P0009r1 : Polymorphic Multidimensional Array Reference

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0009r1
Date:	2016-02-04
Reply-to:	hcedwar@sandia.gov, balelbach@lbl.gov
Author:	H. Carter Edwards
Contact:	hcedwar@sandia.gov
Author:	Bryce Lelbach
Contact:	balelbach@lbl.gov
Author:	Christian Trott
Contact:	crtrott@sandia.gov
Author:	Juan Alday
Contact:	juanalday@gmail.com
Author:	Jesse Perla
Contact:	jesse.perla@ubc.ca
Author:	Mauro Bianco
Contact:	mbianco@cscs.ch
Author:	Robin Maffeo
Contact:	Robin.Maffeo@amd.com
Author:	Ben Sander
Contact:	ben.sander@amd.com
Audience:	Library Evolution Working Group (LEWG)
Audience:	Evolution Working Group (EWG) for concise array declaration
URL:	https://github.com/kokkos/PolyView/blob/master/P0009.rst

1 Rationale for polymorphic multidimensional array reference

Multidimensional arrays are a foundational data structure for science and engineering codes, as demonstrated by their extensive use in FORTRAN for five decades. A *multidimensional array reference* is a reference to a memory extent through a **layout** mapping from a multi-index space (domain) to that extent (range). A array layout mapping may be bijective as in the case of a traditional multidimensional array, injective as in the case of a subarray, or surjective to express symmetry.

Traditional layout mappings have been specfied as part of the language. For example, FORTRAN specifies *column major* layout and C specifies *row major* layout. Such a language-imposed specification requires signficant code refactoring to change an array's layout, and requires significant code complexity to implement non-traditional layouts such as tiling in modern linear algebra or structured grid application domains. Such layout changes are required to adapt and optimize code for varying computer architectures; for example, to change a code from *array of structures* to *structure of arrays*. Furthermore, multiple versions of code must be maintained for each required layout.

A multidimensional array reference abstraction with a polymorphic layout is required to enable changing array layouts without extensive code refactoring and maintenance of functionally redundant code. Layout polymorphism is a critical capability; however, it is not the only beneficial form of polymorphism.

The Kokkos library (github.com/kokkos/kokkos) implements multidimensional array references with polymorphic layout, and other access properties as well. Until recently the Kokkos implementation was limited to C++1998 standard and is incrementally being refactored to C++2011 standard. Additionally, there is a standalone reference implementation of this proposal which is publicly available on github (github.com/brycelelbach/array_ref).

2 Extensibility beyond multdimensional array layout

The polymorphic **array_ref** abstraction and interface has utility well beyond the multidimensional array layout property. Other planned and prototyped properties include specification of which *memory space* within a heterogeneous memory system the referenced data resides on and algorithmic access intent properties. Examples of access intent properties include

1. *read-only random with locality* such that member queries are performed through GPU texture cache hardware for GPU memory spaces,
2. *atomic* such that member access operations are overloaded via proxy objects to atomic operations (see P0019, Atomic View),
3. *non-temporal* such that member access operations can be overloaded with non-caching reads and writes, and
4. *restrict* to guarantee non-aliasing of referenced data within the current context.

3 Compare and contrast summary: previous array_view proposals

The essential issue with **array_view** in N4512 "Multidimensional bounds, offset and array_view, revision 7" is that it did not fulfill C++'s *zero-overhead abstraction* requirement (for both dynamic and static extents), and does not provide a zero-overhead abstraction to different memory layouts which are essential for library interoperability with a variety of C++ (e.g. Eigen) and other languages (e.g. Fortran and Matlab's C++ interface). Were it to be accepted, another library would be necessary to provide "direct mapping to the hardware" for views of arrays. Many of the issues are discussed in more detail in N4355, N4300, and N4222.

Unlike N4512, this proposal

- allows the layout to be more general with different orderings and padding essential for performant member access (*e.g.*, caching, coalescing),
- enables interoperability with libraries using compile-time extents, and
- achieves zero-overhead abstraction for *constexpr* extents and strides which provides more opportunities for the compiler to optimize array data member access functions.

The P0122 "array_view: bounds-safe views for sequences of objects" proposal is largely compatible with this proposal but only supports 1-dimensional array references or layout polymorphism, and thus does not address the entire problem space.

4 Array Reference

The proposed **array_ref** has template arguments for the data type of the array and a parameter pack for polymorphic properties of the reference.

```
namespace std {
namespace experimental {
    template< typename DataType , typename ... Properties >
    struct array_ref ;
}}
```

The complete proposed specification for **array_ref** is included at the end of this paper. We present the specification incrementally to convey the rational for this specification.

An initial set of properties are proposed. These properties are defined by class types and reside in the **array_property** namespace, similar to the namespaces for **std::rel_ops** functions, **std::chrono** classes, or **std::regex_constants** constants.

```
namespace std {
namespace experimental {
namespace array_property {
    // array property classes
}}}
```

5 One-Dimensional Array

A reference to a one-dimension array is anticipated to subsume the functionality of a pointer to a memory extent combined with an array length. For example, a one-dimensional array is passed to a function as follows.

```
void foo( int A[] , size_t N ); // Traditional API
void foo( const int A[] , size_t N ); // Traditional API

void foo( array_ref< int[] > A ); // Reference API
void foo( array_ref< const int[] > A ); // Reference API

void bar()
```

```
{
  enum { L = ... };
  int buffer[ L ];
  array_ref<int[]> A( buffer , L );

  assert( L == A.size() );
  assert( & A[0] == buffer );

  foo( array );
}
```

The *const-ness* of an **array_ref** is analogous to the *const-ness* of a pointer. A **const array_ref<D>** is similar to a const-pointer in that the **array_ref** may not be modifid but the referenced extent of memory may be modified. A **array_ref<const D>** is similar to a pointer-to-const in that the referenced extent of memory may not be modified. These are the same *const-ness* semantics of **unique_ptr** and **shared_ptr**.

The **T[]** syntax has precedence in the standard; **unique_ptr** supports this syntax to denote a **unique_ptr** which manages the lifetime of a dynamically allocated array of objects.

6 Traditional Multidimensional Array with Explicit Dimensions

A traditional multidimensional array with explicit dimensions (for example, an array of 3x3 tensors) is passed to a function as follows.

```
void foo( double A[][3][3] , size_t N0 ); // Traditional API
void foo( array_ref< double[][3][3] > A ); // Reference API

void bar()
{
  enum { L = ... };
  int buffer[ L * 3 * 3 ];
  array_ref< double[][3][3] > A( buffer , L );

  assert( 3 == A.rank() );
  assert( L == A.extent(0) );
  assert( 3 == A.extent(1) );
  assert( 3 == A.extent(2) );
  assert( A.size() == A.extent(0) * A.extent(1) * A.extent(2) );
  assert( & A(0,0,0) == buffer );

  foo( A );
}
```

7 Multidimensional Array with Multiple Implicit Dimensions

The current multidimensional array type declaration in **n4567 8.3.4.p3** restricts array declarations such that only the leading dimension may be implicit. Multidimensional arrays with multiple implicit dimensions as well as explicit dimensions are supported with the **dimension** property. The dimension property uses the "magic value" zero to denote an implicit dimension. The "magic value" of zero is chosen for consistency with **std::extent**.

```
array_ref< int[][3] > x ;

assert( x.extent(0) == 0 );
assert( x.extent(1) == 3 );

assert( extent< int[][3] , 0 >::value == 0 );
assert( extent< int[][3] , 1 >::value == 0 );

array_ref< int , array_property::dimension<0,0,3> > y ;
assert( y.extent(0) == 0 );
assert( y.extent(1) == 0 );
assert( y.extent(2) == 3 );

array_ref< int , array_proprety::dimension<0,0,3> > z(ptr,N0,N1);
assert( z.extent(0) == N0 );
assert( z.extent(1) == N1 );
assert( z.extent(2) == 3 );
```

7.1 Preferred Syntax

We prefer the following concise and intuitive syntax for arrays with multiple implict dimensions.

```
array_ref< int[][][3] > y ; // concise intuitive syntax
```

However, this syntax requires a relaxation of the current multidimensional array type declaration in **n4567 8.3.4.p3**. Furthermore, this concise and intuitive syntax eliminates the need for `array_property::dimension<...>` and the associated "magic value" of zero to denote an implicit dimension.

8 Array Reference Properties: Layout Polymorphism

The **array_ref::operator()** maps the input multi-index from the array's cartesian product multi-index *domain* space to a member in the array's *range* space. This is the **layout** mapping for the referenced array. For natively declared multidimensional arrays the layout mapping is defined to conform to treating the multidimensional array as an *array of arrays of arrays ...*; i.e., the size and span are equal and the strides increase from right-to-left (the layout specified in the C language). In the FORTRAN language defines layout mapping with strides increasing from left-to-right. These *native* layout mappings are only two of many possible layouts. For example, the *basic linear algebra subprograms (BLAS)* standard defines dense matrix layout mapping with padding of the leading dimension, requiring both dimensions and **LDA** parameters to fully declare a matrix layout.

A property template parameter specifies a layout mapping. If this property is omitted the layout mapping of the array reference conforms to a corresponding natively declared multidimensional array as if implicit dimensions were declared explicitly. The default layout is *regular* - the distance is constant between entries when a single index of the multi-index is incremented. This distance is the *stride* of the corresponding dimension. The default layout mapping is bijective and the stride increases monotonically from the right most to the left most dimension.

```
// The default layout mapping of a rank-four multidimensional
// array is as if implemented as follows.

template< size_t N0 , size_t N1 , size_t N2 , size_t N3 >
size_t native_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
  {
    return i0 * N3 * N2 * N1 // stride == N3 * N2 * N1
         + i1 * N3 * N2      // stride == N3 * N2
         + i2 * N3           // stride == N3
         + i3 ;              // stride == 1
  }
```

An initial set of layout properties are **layout_right**, **layout_left**, **layout_order**, and **layout_stride**,

```
namespace std {
namespace experimental {
namespace array_property {
    struct layout_right ;
    struct layout_left ;
    template< unsigned ... > struct layout_order ;
    struct layout_stride ;
}}}
```

```
typedef array_ref< int , array_property::dimension<0,0,3> > array_native ;

typedef array_ref< int , array_property::dimension<0,0,3>
                       , array_property::layout_right > array_right ;

typedef array_ref< int , array_property::dimension<0,0,3>
                       , array_property::layout_left >  array_left ;

assert( std::is_same< typename array_native::layout , void >::value );
assert( std::is_same< typename array_right ::layout ,
                      array_property::layout_right >::value );
assert( std::is_same< typename array_left  ::layout ,
                      array_property::layout_left >::value );

assert( array_native::is_regular::value );
assert( array_right ::is_regular::value );
assert( array_left  ::is_regular::value );
```

A **void** (*a.k.a.*, default or native) mapping is regular and bijective with strides increasing from increasing from right most to left most dimension. A **layout_right** mapping is regular and injective (may have padding) with strides increasing from right most to

left most dimension. A **layout_left** mapping is regular and injective (may have padding) with strides increasing from left most to right most dimension. A **layout_order** mapping is regular and injective (may have padding) with stride ordering defined by the template parameter pack. A **layout_stride** mapping is regular; however, it might not be injective or surjective.

```
// The right and left layout mapping of a rank-four
// multidimensional array could be is as if implemented
// as follows.  Note that padding is allowed but not required.

template< size_t N0 , size_t N1 , size_t N2 , size_t N4 >
size_t right_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
  {
    const size_t S3 = // stride of dimension 3
    const size_t P3 = // padding of dimension 3
    const size_t P2 = // padding of dimension 2
    const size_t P1 = // padding of dimension 1
    return i0 * S3 * ( P3 + N3 ) * ( P2 + N2 ) * ( P1 + N1 )
         + i1 * S3 * ( P3 + N3 ) * ( P2 + N2 )
         + i2 * S3 * ( P3 + N3 )
         + i3 * S3 ;
  }

template< size_t N0 , size_t N1 , size_t N2 , size_t N4 >
size_t left_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
  {
    const size_t S0 = // stride of dimension 0
    const size_t P0 = // padding of dimension 0
    const size_t P1 = // padding of dimension 1
    const size_t P2 = // padding of dimension 2
    return i0 * S0
         + i1 * S0 * ( P0 + N0 )
         + i2 * S0 * ( P0 + N0 ) * ( P1 + N1 )
         + i3 * S0 * ( P0 + N0 ) * ( P1 + N1 ) * ( P2 + N2 );
  }
```

9 Array Reference Properties: Extensible Layout Polymorphism

The **array_ref** is intended to be extensible such that a user may supply a customized layout mapping. A user supplied customized layout mapping will be required to conform to a specified interface; *a.k.a.*, a C++ Concept. Details of this extension point will be included in a subsequent proposal. Our current extensibility strategy is for a user supplied layout property to contain an offset mapping as illustrated below.

```
struct layout_tile_left {
  template< typename Dimension > struct offset ;
};
```

Motivation: An important customized layout mapping is hierarchical tiling. This kind of layout mapping is used in dense linear algebra matrices and computations on Cartesian grids to improve the spatial locality of array entries. These mappings are bijective but are not regular. Computations on such multidimensional arrays typically iterate through tiles as *subarray* of the array.

```
template< size_t N0 , size_t N1 , size_t N2 >
size_t tiling_left_mapping( size_t i0 , size_t i1 , size_t i2 )
{
  static constexpr size_t T = // cube tile size
  constexpr size_t T0 = ( N0 + T - 1 ) / T ; // tiles in dimension 0
  constexpr size_t T1 = ( N1 + T - 1 ) / T ; // tiles in dimension 1
  constexpr size_t T2 = ( N2 + T - 1 ) / T ; // tiles in dimension 2

  // offset within tile + offset to tile
  return ( i0 % T ) + T * ( i1 % T ) + T * T * ( i2 % T )
       + T * T * T * ( ( i0 / T ) + T0 * ( ( i1 / T ) + T1 * ( i2 / T ) ) );
}
```

Note that a tiled layout mapping is irregular and if padding is required to align with tile boundarries then the span will exceed the size. A customized layout mapping will have slightly different requirements depending on whether the layout is regular or irregular.

10 Array Reference Properties: Flexibility and Extensibility

One or more array properties of **void** are acceptable and have no effect. This allows user code to define a template argument list of potential array properties and then enable/disable a particular property by conditionally setting it to **void**. For example:

```
using layout = std::conditional<
    ALLOW_PADDING , array_property::layout_right , void
>::type ;

// If ALLOW_PADDING then use layout_right else use native layout
typedef array< int , array_property::dimension<0,0,3> , layout > MyType ;
```

11 Specification with Simple Array Reference Properties

Simple array properties include the array layout and if necessary a **array_property::dimension** type for arrays with multiple implicit dimensions. Array reference properties are provided through a variadic template to support extensibility of the array reference. Possible additional properties include array bounds checking, atomic access to members, memory space within a heterogeneous memory architecture, and user access pattern hints.

```
namespace std {
namespace experimental {

template< typename DataType , typename ... Properties >
struct array_ref {
  //--------------------
  // Types:

  // Types are implementation and Properties dependent.
  // The following type implementation are normative
  // with respect to empty Properties.

  using value_type = typename std::remove_all_extents< DataType >::type ;
  using reference  = value_type & ; // Typical type, but implementation defined
  using pointer    = value_type * ; // Typical type, but implementation defined
  using size_type  = size_t ; // Typical type, but implementation defined

  using layout = *array layout type* ;

  //--------------------
  // Domain index space properties:

  static constexpr size_type rank() noexcept ;

  template< typename IntegralType >
  constexpr size_type extent( IntegralType index ) const noexcept ;

  // Cardinality of index space; i.e., product of extents
  constexpr size_type size() const noexcept ;

  //--------------------
  // Layout mapping properties:

  static constexpr bool is_regular() noexcept ;

  // If the layout mapping is regular then return the
  // distance between members when index \# is increased by one.
  template< typename IntegralType >
  constexpr size_type stride( IntegralType index ) const noexcept ;

  // Span covering the members
  constexpr size_type span() const noexcept ;

  // Span of an array with regular layout if it
  // is constructed with the given implicit dimensions.
  template< typename ... IntegralArgs >
  static constexpr size_type span( IntegralArgs ... implicit_dims ) noexcept;

  // Pointer to member memory
  constexpr pointer data() const noexcept ;

  //--------------------
  // Member access

  template< typename ... IntegralArgs >
  reference operator()( IntegralArgs ... indices ) const noexcept ;

  template< typename IntegralType >
```

```
  reference operator[]( IntegralType index ) const noexcept ;

  //--------------------
  // Construct/move/copy/destroy:

  ~array_ref() ;

  constexpr array_ref() noexcept ;

  constexpr array_ref( array_ref && rhs ) noexcept = default ;
  constexpr array_ref( const array_ref & rhs ) noexcept = default ;
  array_ref & operator = ( array_ref && rhs ) noexcept = default ;
  array_ref & operator = ( const array_ref & rhs ) noexcept = default ;

  template< typename ... IntegralArgs >
  constexpr array_ref( pointer , IntegralArgs ... implicit_dims ) noexcept ;

  template< typename UType , typename ... UProperties >
  constexpr array_ref( const array_ref< UType , UProperties ... > & rhs ) noexcept ;

  template< typename UType , typename ... UProperties >
  array_ref & operator = ( const array_ref< UType , UProperties ... > & rhs ) noexcept ;
};

}}

namespace std {
namespace experimental {
namespace array_property {

struct layout_right ;
struct layout_left ;
template< unsigned ... > layout_order ;
struct layout_stride ;

template< class T > struct is_layout ;
template< class T > constexpr bool is_layout_v = is_layout<T>::value ;

template< class T > struct is_regular ;
template< class T > constexpr bool is_regular_v = is_regular<T>::value ;

constexpr size_t maximum_rank = /* >= 10 */ ;

// Specify mix of explicit and implict dimensions
template< size_t ... >
struct dimension {
  //--------------------
  // Types:

  using value_type = size_t ; // Typical type, but implementation defined

  //--------------------
  // Construct/move/copy/destroy:

  ~dimension() = default ;

  constexpr dimension() noexcept ;

  constexpr dimension( dimension && rhs ) noexcept = default ;
  constexpr dimension( const dimension & rhs ) noexcept = default ;
  dimension & operator = ( dimension && rhs ) noexcept = default ;
  dimension & operator = ( const dimension & rhs ) noexcept = default ;

  static constexpr size_type rank() noexcept ;

  //--------------------
  // Member access

  template< typename IntegralType >
  constexpr value_type operator[]( IntegralType dim ) const noexcept ;
};

}}}
```

Properties template parameter pack

An array_ref is given properties (such as `layout_left`, `layout_right`, and `layout_stride`) through the `Properties` template parameter pack.

Effects: A `void` member in a `Properties` pack is ignored.

Layout Properties

If a layout property does not appear in the `Properties` pack the layout is `void`.

Requires: is_layout_v< void > == true, is_layout_v< layout_right > == true, is_layout_v< layout_left > == true, is_layout_v< layout_stride > == true, and is_layout_v< layout_order<...> > == true.

Requires: is_regular_v< void > == true, is_regular_v< layout_right > == true, is_regular_v< layout_left > == true, is_regular_v< layout_stride > == true, and is_regular_v< layout_order<...> > == true.

Requires: At most one member of the `Properties` pack is a layout property.

Effects: `array_ref::layout` is the layout property given in the `Properties` pack or `void` if no layout property is given.

Effects: Given a one of these regular layouts an array_ref strides and layout mapping conform to the following.

```
using a_type = array_ref<DataType,Properties...> ;

a_type a( ptr , dims... );

if ( std::is_lvalue_reference_v< a_type::reference > && a_type::is_regular
   && 0 <= i && i <= a_type::rank() && 0 <= ji && ji < a.extent(i) - 1 ) {
  assert( std::distance( & a(j0,...,ji,...) , & a(j0,...,ji+1,...) == a.stride(i) );
}

if ( std::is_same_v< a_type::layout , void > ) {
  assert( a_type::is_regular );
  if ( i + 1 == a_type::rank() )
    assert( a.stride(i) == 1 );
  else if ( 0 < i )
    assert( a.stride(i) == a.stride(i+1) * a.dimension(i+1) );
}
else if ( std::is_same_v< a_type::layout , layout_right > ) {
  assert( a_type::is_regular )
  if ( i + 1 == a_type::rank() )
    assert( a.stride(i) == 1 );
  else if ( 0 < i )
    assert( a.stride(i) >= a.stride(i+1) * a.dimension(i+1) );
}
else if ( std::is_same_v< a_type::layout , layout_left > ) {
  assert( a_type::is_regular )
  if ( i == 0 && 0 < a.rank() )
    assert( a.stride(i) == 1 );
  else if ( i < a_type::rank() )
    assert( a.stride(i) >= a.stride(i-1) * a.dimension(i-1) );
}
```

array_property::layout_order< unsigned ... order >

Requires: The members of `order` are the integers [0 , `sizeof...(order)`).

Requires: When an `array_ref` has a `layout_order` property then `rank() == sizeof...(order)`.

Effects: The `order` pack specifies the ordering relationship of dimensions in the mapping.

```
constexpr unsigned i0 = /* [0, 3) */ ;
constexpr unsigned i1 = /* [0, 3)  and  != i1 */ ;
constexpr unsigned i2 = /* [0, 3)  and  != i1  and  != i2 */ ;

using a_type = array_ref<int[][3][4] , layout_order< i0 , i1 , i2 > > ;

assert( a_type::is_regular );

a_type A( ptr , dims... );

assert( a.stride(i0) == 1 );
assert( a.stride(i1) >= a.stride(i0) * a.extent(i0) );
assert( a.stride(i2) >= a.stride(i1) * a.extent(i1) );
```

Dimension Specification

The dimension specification of an array_ref may be given through the `DataType` template argument or through one of the `Properties` template arguments. For example, the dimension specification for an array_ref with leading implicit dimension and a

second explicit dimension is specified by either of the following declarations.

```
array_ref< T [][3] >
array_ref< T , array_property::dimension< 0 , 3 > >
```

Remark: When a dimension specification is part of the `DataType` the specification is limited by the valid *multidimensional array type* declaration syntax (n4567 8.3.4.p3). If a relaxation of the current multidimensional array type declaration were made the `array_property::dimension< ... >` would be unnecessary and eliminated from this proposal.

Requires: If `std::extent< DataType >::value == 0` then at most one `Properties` template argument may be `array_property::dimension< ... >`.

Effects: When the dimension specification is given as part of the `DataType` then an explicit dimension is specified by each `[N]`, an implicit dimension is specified by each `[]`, `rank() == std::rank< DataType >::value`, and `extent(i) == std::extent< DataType , i >::value` for `i < rank()` and default constructed array_ref.

Effects: When the dimension specification is given via `array_property::dimension< N0, N1 , ... >` then `0 <= Nj` for all j, an explicit dimensions is specified by `0 < Nj` an implicit dimensions is specified by `0 == Nj` `rank() ==` number of arguments, and `extent(j) == Nj` for `j < rank()` and default constructed array_ref.

Requires: `10 <= maximum_rank`

Effect: An implementation supports array references up to maximum_rank.

Remark: An array_ref implementation may use rank-specific optimizations. As such an indefinite maximum rank may be impractical. An implementation must support at least this rank.

using value_type = typename std::remove_all_extents< DataType >::type

The type of each element of the referenced array.

using reference =

The type returned by the member access operator. Typically this will be `value_type &`. [Note: The reference type may be a proxy depending upon the `Properties`. For example, if a property indicates that all member references are to be atomic then the reference type would be a proxy conforming to *atomic-view-concept* introduced in paper P0019. - end note]

using pointer =

The input type to a wrapping constructor.

using size_type =

The type that counts the number of elements in the referenced array.

using layout=

The layout type property that defaults to `void`.

static constexpr size_type rank() noexcept

Returns: The rank of the referenced array.

template< typename IntegralType >

constexpr size_type extent(IntegralType index) const noexcept

Requires: std::is_integral< IntegralType >::value

Returns: When `r < rank()` the exent of dimension, otherwise 1. A default constructed array_ref will have `extent(r) == 0` for all implicit dimensions. The return value of an explicit dimension queried with a literal input value must be "constexpr" observable.

constexpr size_type size() const noexcept

Returns: The product of the extents.

static constexpr bool is_regular() noexcept

Returns: True if the layout mapping is regular; *i.e.*, if there is a uniform stride between members when incrementing a particular dereferencing index and holding all other indices fixed.

template< typename IntegralType >

constexpr size_type stride(IntegralType index) const noexcept

Requires: std::is_integral< IntegralType >::value

Requires: is_regular()

Returns: When is_regular::value and `0 <= r < rank()` the distance between members when index `r` is incremented by one, otherwise 0.

constexpr size_type span() const noexcept

Returns: A distance that is at least maximum distance between any two members of the array plus one. All member of the array reside in the span `[ data() , data() + span() )`.

Remark: For a one-to-one layout mapping the span will equal the size.

template< typename ... IntegralArgs >

static constexpr size_type span(IntegralArgs ... implicit_dims) noexcept

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value`

Requires: All `implicit_dims` parameters are non-negative.

Returns: The span of the array reference if it were constructed with the implicit dimensions.

constexpr pointer data() const noexcept

Requires: All members are in the range `[ data() , data() + span() )`.

Returns: Pointer to the member with the minimum location.

template< typename ... IntegralArgs >

reference operator()(IntegralArgs ... indices) const noexcept

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value`

Requires: All `indices` parameters are non-negative.

Requires: `rank() == sizeof...(IntegralArgs)`

Requires: The ith argument *indices[i]** is in bounds; *indices[i]* `< extent(i)`.

Returns: A reference to the member referenced by the indices argument.

Remark: An implementation may have rank-specific overloads to better enable optimization of the member access operator.

```
template< typename ArrayRefType, typename I0 >
typename std::enable_if< ArrayRefType::rank() == 1
                       , typename ArrayRefType::reference >::type
array_ref_index( I0 i0 ) noexcept ;

template< typename ArrayRefType, typename I0 , typename I1 >
typename std::enable_if< ArrayRefType::rank() == 2
                       , typename ArrayRefType::reference >::type
array_ref_index( I0 i0 , I1 i1 ) noexcept ;

template< typename ArrayRefType, typename I0, typename I1, typename I2
        , typename ... IntegralArgs >
typename std::enable_if< ArrayRefType::rank() == (sizeof...(IntegralArgs) + 3)
                       , typename ArrayRefType::reference >::type
array_ref_index( I0 i0, I1 i1, I2 i2, IntegralArgs ... indices ) noexcept ;

template< typename DataType , typename ... Properties >
struct array_ref {
   template< typename ... IntegralArgs >
```

```
  reference operator()( IntegralArgs ... indices ) const noexcept {
    return array_ref_index<array_ref>(indices...);
  }
};
```

template< typename IntegralType >

reference operator[](IntegralType index) const noexcept

Requires: rank() == 1

Requires: is_integral< IntegralType >::value

Requires: 0 <= i < extent(0)

Returns: Reference to member denoted by index `i`.

Remark: Provides compatibility with traditional rank-one array member reference.

Remark: It is recommended that the rank and type requirements be enforced by conditionally enabling the operator.

```
template< typename IntegralType >
typename std::enable_if<
    std::is_integral< IntegralType >::value && 1 == rank() , reference
>::type
operator[]( const IntegralType & i ) const noexcept ;
```

~array_ref()

Effect: Assigns `this` to be a *null* array_ref.

Remark: There may be other *property* dependent effects.

constexpr array_ref() noexcept

Effect: Construct a *null* array_ref with extent(i) == 0 for all implicit dimensions and data() == nullptr.

constexpr array_ref(const array_ref & rhs) noexcept = default

Effect: Construct a array_ref of the same array referenced by `rhs`.

Remark: There may be other *property* dependent effects.

array_ref & operator = (const array_ref & rhs) noexcept = default

Effect: Assigns `this` to array_ref the same array referenced by `rhs`.

Remark: There may be other *property* dependent effects.

constexpr array_ref(array_ref && rhs) noexcept = default

Effect: Construct a array_ref of the array referenced by `rhs` and then `rhs` is *null* array_ref.

Remark: There may be other *property* dependent effects.

array_ref & operator = (array_ref && rhs) noexcept = default

Effect: Assigns `this` to array_ref the array referenced by `rhs` then assigns `rhs` to be a *null* array_ref.

Remark: There may be other *property* dependent effects.

template< typename ... IntegralArgs >

constexpr array_ref(pointer ptr , IntegralArgs ... implicit_dims) noexcept

Requires: The input `ptr` references memory `[ ptr , ptr + S )` where S = `array_ref::span(args...)`.

Effects: The *wrapping constructor* constructs a multidimensional array reference of the given member memory such that all data members are in the span `[ ptr , ptr + span() )`.

template< typename UType , typename ... UProperties >

constexpr array_ref(const array_ref< UType , UProperties ... > & rhs) noexcept

Requires: This array_ref type is assignable to the `rhs` array_ref type. Assignability includes compatibility of the value type, dimensions, and properties.

Effect: Constructs a array_ref of the array referenced by `rhs`.

```
array_ref< int[][3] > x(ptr, N0);

// OK: compatible const from non-const and implicit from explicit dimension
array_ref< const int , array_properties::dimension< 0 , 0 > > y(x);

// Error: cannot assign non-const from const
array_ref< int , array_properties::dimension< 0 , 0 > > z(y);
```

template< typename UType , typename ... UProperties >

array_ref & operator = (const array_ref< UType , UProperties ... > & rhs) noexcept

Requires: This array_ref type is assignable to the `rhs` array_ref type.

Effect: Assigns `this` to array_ref the array array_ref by `rhs`.

12 Assignability of Array References of Non-identical Types

It is essential that array_ref of non-identical, compatible types be assignable. For example:

```
array_ref< int[][3] > x(ptr , N0);

// valid assignment
array_ref< const int , array_property::dimension< 0 , 0 > > y(x);
```

The 'std::is_assignable' meta-function must be partial specialized to implement the array_ref assignability rules regarding value type, dimensions, and properties.

```
template< class Utype , class ... Uprop
        , class Vtype , class ... Vprop >
struct is_assignable< array_ref< Utype , Uprop ... >
                    , array_ref< Vtype , Vprop ... > >
  : public integral_const< bool ,
      is_assignable< typename array_ref< Utype , Uprop ... >::pointer
                   , typename array_ref< Vtype , Vprop ... >::pointer
                   >::value
      &&
      ( array_ref< Utype , Uprop ... >::rank() ==
        array_ref< Vtype , Vprop ... >::rank() )
      &&
      (
        // Extent is either equal or implicit.
        extent<Utype,#>::value == extent<Vtype,#>::value ||
        extent<Utype,#>::value == 0
      )
      &&
      // other possible conditions
      > {}
```

Assignability extends beyond the **cv** qualification of the array_ref's data. For example,

1. Implicitly dimensioned array_ref are assignable from equal rank explicitly dimensioned array_ref,
2. Strided layout array_ref with implicit dimensions are assignable from equal rank array_ref with regular layout, or
3. A array_ref with an access intent property, such as *random* or *restrict* may be assigned from a array_ref without such a property.

13 Subarray Interface

The capability to **easily** extract subarrays of an array, or subarrays of subarrays, is essential for usability. Non-trivial subarrays of regular arrays will often have **layout_stride**.

```
using U = array_ref< int , array_properties::dimension<0,0,0> > ;

U x(buffer,N0,N1,N2);
```

```
// Using std::pair<int,int> for an integral range
auto y = subarray( x , std::pair<int,int>(1,N0-1) ,
                      std::pair<int,int>(1,N1-1) , 1 );

assert( y.rank() == 2 );
assert( y.extent(0) == N0 - 2 );
assert( y.extent(0) == N1 - 2 );
assert( & y(0,0) == & x(1,1,1) );

// Using initializer_list of size 2 as an integral range
auto z = subarray( x , 1 , {1,N1-1} , 1 );

assert( z.rank() == 1 );
assert( & z(0) == & x(1,1,1) );

// Conveniently extracting subarray for all of a extent
// without having to explicitly extract the dimensions.
auto x = subarray( x , array_property::all , 1 , 1 );
```

`subarray()` returns an unspecified instantiation of `array_ref<>`. There is precedence in the standard for library functions with unspecified return types (e.g. `bind()`).

```
namespace std {
namespace experimental {
namespace array_property {

struct all_type {};
constexpr all_type all = all_type();

}

template< typename DataType , typename ... Properties ,
          typename ... SliceSpecifiers >
/*unspecified array_ref<>*/
subarray( const array_ref< DataType, Properties ... > & ar ,
          SliceSpecifiers ... specs) noexcept;

template< typename DataType , typename ... Properties ,
          typename ... SliceSpecifiers >
/*unspecified array_property::dimension<>*/
subdimensions( const array_ref< DataType, Properties ... > & ar ,
               SliceSpecifiers ... specs) noexcept;

template< typename DataType , typename ... Properties ,
          typename ... StrideSpecifiers >
/*unspecified array_ref<>*/
stridearray( const array_ref< DataType, Properties ... > & ar ,
             StrideSpecifiers ... specs) noexcept;

}}
```

template< typename T , typename ... Properties , typename ... SliceSpecifiers >

/*unspecified array_ref<>*/

subarray(const array_ref< T, Properties ... > & ar , SliceSpecifiers ... specs) noexcept

> Requires: `sizeof...(Properties) == sizeof...(SliceSpecifiers)`
>
> Requires: Each SliceSpecifier parameter must either be an integral type T, a pair<T, T>, a tuple<T, T>, an initializer_list<T> with a size of 2, or an array<T, 2>.
>
> Requires: A SliceSpecifier parameter for dimension `i` which expresses a fixed index indices must be within the range `[ 0 , ar.extent(i) )`.
>
> Requires: A SliceSpecifier parameter for dimension `i` which expresses an interval of indices must be a subset of the range `[ 0 , ar.extent(i) )`.
>
> Returns: An array_ref<> referring to the same memory extent as the array_ref<> `ar`, with dimensions and layout specified by the SlicerSpecifier parameters.
>
> Remarks: For each SliceSpecifier parameter which is an integer value, the produced array_ref<> will omit the corresponding dimension. E.g. the produced array_ref<>'s rank will be is one less than the rank of `ar`.

```
assert(subarray(ar, x, array_property::all)(y) == ar(x, y));
```

```
assert(subarray(ar, array_property::all, y)(x) == ar(x, y));

assert(subarray(ar, array_property::all, y, array_property::all)(x, z) == ar(x, y, z));
```

Remarks: For each SliceSpecifier parameter which expresses an interval of indices `[ a , b )`, the produced array_ref<> will have extent `[ 0 , b - a )` in the corresponding dimension.

```
// Ok
assert(subarray(ar, {10, 20})(0)  == ar(10));
assert(subarray(ar, {10, 20})(10) == ar(20));

// Undefined behavior: subarray is out of bounds.
assert(subarray(ar, {10, 20})(20) == ar(20));
```

template< typename T , typename ... Properties , typename ... SliceSpecifiers >

/*unspecified array_property::dimension<>*/

subdimensions(const array_ref< T, Properties ... > & ar , SliceSpecifiers ... specs) noexcept

Returns: An array_property::dimension<> object referring to the dimensions of the array_ref<> that would be constructed by calling subarray() on `ar` with SliceSpecifiers `specs`.

template< typename T , typename ... Properties , typename ... StrideSpecifiers >

/*unspecified array_ref<>*/

stridearray(const array_ref< T , Properties ... > & ar , StrideSpecifiers ... specs) noexcept

Requires: `sizeof...(Properties) == sizeof...(StrideSpecifiers)`

Requires: `conjunction<is_integral<StrideSpecifiers>::type...>::value`

Requires: A StrideSpecifier parameter for dimension `i` must be within the range `[ 1 , ar.extent(i) )`.

Returns: An array_ref<> referring to the same memory extent as the array_ref<> `ar` with the same rank as `ar`, and dimensions and layout specified by the StrideSpecifier parameters.

Remarks: The stride of the produced array_ref<> for a given dimension will be the corresponding StrideSpecifier parameter.

14 Limited Iterator Interface

A **array_ref** may have a non-isomorphic mapping between its multi-index space (domain) and span of member memory (range). For example, a subarray or dimension padded array will be non-isomorphic. An iterator for the members of a non-isomorphic array_ref must be non-trivial in order to skip over non-member spans of memory. Thus a general iterator implementation would necessarily be non-trivial both in state and algorithm. As such we provide a very limited iterator interface conforming to **24.6.5 range access** for a rank-one array with isomorphic layout (*e.g.*, default, **layout_left**, **layout_right**) and no incompatible access intent properties (*e.g.*, the **reference** type is truly a reference and not a proxy). For example, a simple **array_ref<T[]>** will have **begin** and **end** overloads.

```
namespace std {

template< class T , class ...P >
// Enabled if rank one and isomorphic layout and no incompatible access
// intent properties.
typename std::enable_if< /*unspecified*/
                       , typename array_ref<T,P...>::pointer >::type
begin( const std::experimental::array_ref<T,P...> & v )
{ return v.data(); }

template< class T , class ...P >
// Enabled if rank one and isomorphic layout and no incompatible access
// intent properties.
typename std::enable_if< /*unspecified*/
                       , typename array_ref<T,P...>::pointer >::type
end( const std::experimental::array_ref<T,P...> & v )
{ return v.data() + v.size(); }

}
```

Note that in the more general case of an isomorphic array_ref of any rank a pointer (iterator) range for array_ref member data can be queried.

```
template< typename T , class ... P >
void foo( array_ref<T,P...> a )
{
  if ( std::is_reference< typename array_ref<T,P...>::reference >::value
       && a.size() == a.span() )
  {
    // Iteration via pointer type is valid and performant
    typename array_ref<T,P...>::pointer
      begin = a.data() ,
      end   = a.data() + a.span() ;
  }
}
```

15 Example Usage in an 8th Order Finite Difference Stencil

The subarray interface provides a powerful mechanism for accessing 3-dimensional data in numerical kernels in a fashion which utilizes performant memory access patterns and is amenable to compiler-assisted vectorization.

The following code is an example of a typical finite difference stencil which might be used in a computational fluid dynamics application. This code utilizes operator splitting to avoid vector register pressure and moves through memory in unit stride to facilitate optimal memory access patterns. With the addition of compiler alignment hints (as well as padding and aligned allocations to make those assumptions true) and compiler directives or attributes to indicate that the input pointers do not alias each other, this code would vectorize well on a traditional x86 platform.

```
void eighth_order_stencil(
  const double* V, double* U,
  ptrdiff_t dx, ptrdiff_t dy, ptrdiff_t dz,
  array<double, 5> c)
{
  // Iterate over interior points, skipping the 4 cell wide ghost
  // zone region.
  for (int iz = 4; iz < dz - 4; ++iz)
    for (int iy = 4; iy < dy - 4; ++iy) {
      // Pre-compute shared iy and iz indexing to ensure redundant
      // calculations are avoided.
      double const* v = &V[iy*dx + iz*dx*dy];
      double*       u = &U[iy*dx + iz*dx*dy];

      // X-direction (unit stride) split.
      for (int ix = 4; ix < dx - 4; ++ix)
        u[ix] =  c[0] * v[ix]
              +  c[1] * (v[ix+1] + v[ix-1])
              +  c[2] * (v[ix+2] + v[ix-2])
              +  c[3] * (v[ix+3] + v[ix-3])
              +  c[4] * (v[ix+4] + v[ix-4]);

      // Y-direction (dx stride) split.
      for (int ix = 4; ix < dx - 4; ++ix)
        u[ix] += c[1] * (v[ix+dx]   + v[ix-dx])
              +  c[2] * (v[ix+2*dx] + v[ix-2*dx])
              +  c[3] * (v[ix+3*dx] + v[ix-3*dx])
              +  c[4] * (v[ix+4*dx] + v[ix-4*dx]);

      // Z-direction (dx*dy stride) split.
      for (int ix = 4; ix < dx - 4; ++ix)
        u[ix] += c1 * (v[ix+dx*dy]   + v[ix-dx*dy])
              +  c2 * (v[ix+2*dx*dy] + v[ix-2*dx*dy])
              +  c3 * (v[ix+3*dx*dy] + v[ix-3*dx*dy])
              +  c4 * (v[ix+4*dx*dy] + v[ix-4*dx*dy]);
    }
}
```

The corresponding code can be rewritten using array_ref<> and the associated subarray() interfaces.

```
void eighth_order_stencil(
  array_ref<const double, array_property::dimension<0, 0, 0>, /*...*/> V,
  array_ref<double, array_property::dimension<0, 0, 0, /*...*/> U,
  array<double, 5> c)
{
  // Compute the dimensions of the interior region.
  auto interior = subdimensions(V, {4, V.extent(0)-4},
```

```
                                {4, V.extent(1)-4},
                                {4, V.extent(2)-4});

  for (int iz = 0; iz < interior[2]; ++iz)
    for (int iy = 0; iy < interior[1]; ++iy) {
      // Pre-compute shared iy and iz indexing to ensure redundant
      // calculations are avoided.
      auto v = subarray(V, {4, V.extent(0)-4}, iy, iz);
      auto u = subarray(U, {4, V.extent(0)-4}, iy, iz);

      // X-direction (unit stride) split.
      for (int ix = 0; ix < interior[0]; ++ix)
        u[ix] =  c[0] * v[ix]
              +  c[1] * (v[ix+1] + v[ix-1])
              +  c[2] * (v[ix+2] + v[ix-2])
              +  c[3] * (v[ix+3] + v[ix-3])
              +  c[4] * (v[ix+4] + v[ix-4]);

      // Y-direction (dx stride) split.
      v = stridearray(v, V.stride(1));
      u = stridearray(u, U.stride(1));
      for (int ix = 0; ix < interior[0]; ++ix)
        u[ix] += c[1] * (v[ix]   + v[ix])
              +  c[2] * (v[ix+2] + v[ix-2])
              +  c[3] * (v[ix+3] + v[ix-3])
              +  c[4] * (v[ix+4] + v[ix-4]);

      // Z-direction (dx*dy stride) split.
      v = stridearray(v, V.stride(2));
      u = stridearray(u, U.stride(2));
      for (int ix = 0; ix < interior[0]; ++ix)
        u[ix] += c[1] * (v[ix]   + v[ix])
              +  c[2] * (v[ix+2] + v[ix-2])
              +  c[3] * (v[ix+3] + v[ix-3])
              +  c[4] * (v[ix+4] + v[ix-4]);
    }
}
```

Slicing down to a 1-dimensional array_ref<> in the inner-most unit-stride loops also facilitates the use of the 1-dimensional iterator interface, enabling interoperability with iterator-based algorithms.

16 Array Reference Property : Member Access Array Bounds Checking

```
namespace std {
namespace experimental {
namespace array_property {

struct bounds_checking ;

}}}
```

Array bounds checking is an invaluable tool for debugging user code. This functionality traditionally requires global injection through special compiler support. In large, long running code global array bounds checking introduces a significant overhead that impedes the debugging process. A member access array bounds checking array property allows the selective injection of array bounds checking and removes the need for special compiler support.

```
// User enables array bounds checking for selected array_ref.

using x_property = typename std::conditional<
    ENABLE_ARRAY_BOUNDS_CHECKING , array_property::bounds_checking , void
>::type ;

array_ref< int , array_property::dimension<0,0,3> , x_property >
    x(ptr,N0,N1);
```

Adding **bounds_checking** to the properties of an array has the effect of introducing an array bounds check to each member access operation. If the requirement `0 <= i# < extent_#()` fails an error message is generated and the member access operator aborts as it is **noexcept**.

17 Preferred Syntax for Multidimensional Array with

Multiple Implicit Dimensions

One goal of the array_ref interface is to preserve syntax between array_ref and arrays with explicit and implicitly declared dimensions. In the following example foo1 and foo2 accept rank 3 arrays of integers with prescribed explicit / implicit dimensions and fooT accepts a rank 3 array of integers with unprescribed dimensions.

```
void foo1( array_ref< int[ ][3][3] > a ); // Two explicit dimensions
void foo2( array_ref< int[ ][ ][ ] > a ); // All implicit dimensions

// Accept a array_ref of a rank three array with value type int
// and dimensions are explicit or implicit.
template< class T , class ... P >
typename std::enable_if< array_ref<T,P...>::rank() == 3 >::type
foo( array_ref<T,P...> a ) { ... }

void bar()
{
  enum { L = ... };
  int buffer[ L * 3 * 3 ];
  array_ref< int[][][] > a( buffer , L , 3 , 3 );

  assert( 3 == a.rank() );
  assert( L == a.extent(0) );
  assert( 3 == a.extent(1) );
  assert( 3 == a.extent(2) );
  assert( a.size() == a.extent(0) * a.extent(1) * a.extent(2) );
  assert( & a(0,0,0) == buffer );

  foo( array );
}
```

17.1 A relaxed multidimensional array type declaration

The current array type declarator constraints are defined in in **n4567 8.3.4.p3** as follows.

> *When several "array of" specifications are adjacent, a multidimensional array type is created; only the first of the constant expressions that specify the bounds of the arrays may be omitted. In addition to declarations in which an incomplete object type is allowed, an array bound may be omitted in some cases in the declaration of a function parameter (8.3.5). An array bound may also be omitted when the declarator is followed by an initializer (8.5). In this case the bound is calculated from the number of initial elements (say, N) supplied (8.5.1), and the type of the identifier of D is "array of N T". Furthermore, if there is a preceding declaration of the entity in the same scope in which the bound was specified, an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.*

The preferred syntax requires a relaxation of array type declarator constraints defined in **n4567 8.3.4.p3** exclusively for an incomplete object type. The following wording change is recommended.

> *When several "array of" specifications are adjacent, a multidimensional array type is created. In declarations in which an incomplete object type is allowed any of the constant expressions that specify bounds of the arrays may be omitted. In some cases in the declaration of a function parameter (8.3.5) the first array bound constant expression may be omitted. The first array bound constant expression may also be omitted when the declarator is followed by an initializer (8.5). In this case the bound is calculated from the number of initial elements (say, N) supplied (8.5.1), and the type of the identifier of D is "array of N T". Furthermore, if there is a preceding declaration of the entity in the same scope in which the bound was specified, the omitted first array bound constant expression is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.*

This minor language specification change has been implemented with a trivial (one line) patch to Clang and was permissible in gcc prior to version 5.